Lesson 8 Teacher's Key: Under Pressure

GDevelop - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Adding a new action to an existing event (the Lesson 7 escape-cleanup event) rather than creating a new event - reinforcing that GDevelop's event-based model lets a single trigger drive multiple, independent consequences cleanly.

Level 1 - Full Worked Solution

- Lives is a new scene variable, parallel to EnemiesRemaining - same scope, same pattern, different name.
- The Lives = Lives - 1 action is added directly to the EXISTING escape-cleanup event from Lesson 7 (Enemy.Y() > 750 -> Delete Enemy), alongside the Delete action - one new action line in an already-correct event.
- A new, separate event (Lives <= 0 -> Show LoseText) mirrors the existing EnemiesRemaining <= 0 -> Show WinText event exactly.

Highlighted lines are new this week; everything else is code you already have.

```
(existing escape event, revised)
Condition: Enemy.Y() > 750
Action: Lives = Lives - 1
Action: Delete Enemy

(new event)
Condition: Lives <= 0
Action: Show LoseText
```

Why This Structure

- GDevelop's per-event action lists make this lesson's core idea very visible: one event (the escape condition) now has two actions (delete the enemy, AND decrement Lives), while the Combat group's collision event remains untouched and still only ever changes EnemiesRemaining.
- This is the direct payoff of good Lesson 7 structure: the escape-cleanup event already existed as a clean, single-purpose event, so this week's addition was one new action line, not a redesign.

Level 2 / Level 3 Talking Points

- Level 3 sound, worked (GDevelop): import both files. In the escape-cleanup event, after Lives = Lives - 1, add Action: Play a sound (sfx_lifelost.wav). In the event with Condition

Lives <= 0, add Action: Play a sound (sfx_gameover.wav). Make the lifelost event only fire when Lives > 0 if students don't want the overlap.

- Ask students to open their event sheet and point to the exact event they added the new action to - confirming they can distinguish it from the Combat group is the real test of understanding, not just getting the game to work.
- Level 3 structure: suggest renaming the escape-cleanup event group to something like "Enemy Movement & Escape" if it isn't already labeled, continuing the organizational habit introduced in Lesson 7.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: shooting an enemy never changes lives (only the kill path
changes enemies_remaining);
// an escape never changes enemies_remaining (only the escape path changes
lives);
// "Game Over" appears once, when lives first reaches 0.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
(existing escape event, Level 3)
Condition: Enemy.Y() > 750
Action: Lives = Lives - 1
Action: Play a sound (sfx_lifelost.wav) - only if Lives > 0, add
Condition: Lives > 0 on a separate event if the overlap bothers students
Action: Delete Enemy

(game over event, Level 3)
Condition: Lives <= 0
Action: Show LoseText
Action: Play a sound (sfx_gameover.wav)

(Level 3 visual)
Action on the escape event: set the content of LivesText (a second Text
object) to "Lives: " + ToString(Lives)
```

Common Student Mistakes

- Adding Lives = Lives - 1 to the Combat group's collision event instead of the escape-cleanup event (makes shooting cost a life, which is backwards).
- Forgetting Lives starts at 3 (scene variables default to 0 unless explicitly set) - check the variable's default value in the Scene Variables editor, not just referenced in an event.
- LoseText and WinText positioned at the same coordinates on the scene, causing them to overlap if both are shown near the same time.